Évaluation et mini-projet final

Le jour qui vérifie que les compétences de la semaine sont réellement acquises : une partie théorique, des exercices pratiques sans modèle à recopier, puis un mini-projet complet publié sur GitHub. Le critère de réussite : expliquer son code, le modifier sans assistance constante et corriger les principales erreurs.

1. Ce qui est évalué
2. Partie 1 · Questions de théorie
3. Partie 2 · Déboguer un programme
4. Partie 2 · Lire un fichier et calculer
5. Partie 3 · Le mini-projet
6. L'architecture du projet
7. La grille de notation
8. Checklist de fin de semaine

1. Ce qui est évalué

PARTIE	CONTENU	DURÉE CONSEILLÉE
1 · Théorie	Expliquer variables, types, conditions, boucles, fonctions, listes, dictionnaires, fichiers, exceptions, modules, terminal et Git	30 min
2 · Pratique	Résoudre sans recopier, déboguer un programme incorrect, lire un fichier et calculer, écrire et tester des fonctions	1 h 30
3 · Mini-projet	Un analyseur de données complet, testé, publié sur GitHub avec un README	3 h

> **Critère de réussite**
>
> Être capable d'**expliquer** chaque ligne de son code, de le **modifier** sans assistance constante et de **corriger** les principales erreurs en lisant les messages de Python.

2. Partie 1 · Questions de théorie

Réponds à voix haute ou par écrit, sans notes. Les réponses attendues sont dans la fiche F2.DAY6.WEEK1 (synthèse).

#	QUESTION
1	Quels sont les 4 types de base ? Pourquoi `"25" + 5` plante-t-il ?
2	Quelle différence entre `=` et `==` ?
3	Quelle différence entre un `elif` et un nouveau `if` ?
4	Comment compter des éléments avec une boucle ? Pourquoi `c = 1` ne marche pas ?
5	Quelle différence entre `return` et `print` ? Que renvoie une fonction sans `return` ?
6	Liste, tuple, dictionnaire, ensemble : lequel choisir, et quand ?
7	Pourquoi `b = a` ne copie-t-il pas une liste ? Comment faire une vraie copie ?
8	À quoi sert `with open` ? Quelle différence entre les modes `"w"` et `"a"` ?
9	Que se passe-t-il dans un `try` quand une erreur survient ?
10	Que fait `import` ? Pourquoi écrire `if __name__ == "__main__":` ?
11	Que font `pwd`, `ls`, `cd ..` ? Qu'est-ce qu'un chemin relatif ?
12	Quelles sont les trois zones de Git ? Que fait `git add`, puis `git commit` ?

3. Partie 2 · Déboguer un programme

Ce programme contient **5 erreurs**. Trouve-les en le lançant et en lisant chaque message, une par une.

```
def moyenne(notes)
    total = 0
    for note in notes:
        total = note
    print(total / len(notes))

notes = ["12", 15, 9]
m = moyenne(notes)
if m >= 10:
    print("Admis")
```

Corrigé ★★

1. `def moyenne(notes)` : il manque `:` → `SyntaxError`.
2. `"12"` est du texte → `TypeError` à l'addition. Il faut `12`.
3. `total = note` efface au lieu d'ajouter (le tableau noir) → `total = total + note`.
4. `print` au lieu de `return` → `m` vaut `None`.
5. Conséquence : `None >= 10` provoque une `TypeError`. Elle disparaît avec la correction 4.

```
def moyenne(notes):
    total = 0
    for note in notes:
        total = total + note
    return total / len(notes)

notes = [12, 15, 9]
m = moyenne(notes)
if m >= 10:
    print("Admis")
```

---

4. Partie 2 · Lire un fichier et calculer

> **Exercice**
>
> Écrire `frequences(chemin)` qui lit `WEEK1/DAY1/exercices/resultats.txt` (10 000 thèmes, avec des majuscules, des espaces en trop et des lignes vides), nettoie chaque ligne (`strip`, `lower`), ignore les lignes vides et renvoie un `Counter`. Afficher ensuite le top 3.

Corrigé ★★

```
from collections import Counter

def frequences(chemin):
    compte = Counter()
    with open(chemin, encoding="utf-8") as f:
        for ligne in f:
            theme = ligne.strip().lower()
            if theme != "":
                compte[theme] += 1
    return compte

print(frequences("resultats.txt").most_common(3))
```

---

5. Partie 3 · Le mini-projet

Cahier des charges : l'analyseur de thèmes d'examen

Un programme `analyseur_themes`, dans son propre dépôt GitHub, qui :

1. **lit** des données depuis un fichier dont le chemin est donné en argument (`python src/main.py data/resultats.txt`) ;
2. **nettoie** les données : espaces, majuscules, lignes vides ;
3. **calcule** des statistiques : nombre total, nombre de thèmes distincts, pourcentage de chaque thème ;
4. **identifie** les 5 valeurs les plus fréquentes ;
5. **gère les erreurs** : fichier manquant, argument oublié, fichier vide ;
6. **affiche** un rapport clair et le sauvegarde en JSON ;
7. contient des **tests** pytest ;
8. est **publié sur GitHub** avec un README.

6. L'architecture du projet

```
analyseur-themes/
├── data/resultats.txt          # copie du fichier de 10 000 lignes
├── src/
│   ├── lecture.py              # lire_lignes(chemin), nettoyer(lignes)
│   ├── stats.py                # compter(themes), top(compte, k), pourcentages(c
│   ├── rapport.py              # afficher(...), sauver_json(...)
│   └── main.py                 # lit sys.argv, appelle tout, gère les erreurs
├── tests/test_stats.py         # au moins 6 tests : normal, vide, doublons, limi
├── pytest.ini                  # [pytest] pythonpath = src
├── requirements.txt            # pytest
├── .gitignore
└── README.md
```

> **Méthode : l'ordre de construction**
>
> 1. `lecture.py` + ses tests. 2. `stats.py` + ses tests. 3. `rapport.py`. 4. `main.py`. 5. Les cas d'erreur. 6. Le README. 7. Les commits : au moins un par étape, avec des messages clairs. Puis le push.

7. La grille de notation (sur 20)

CRITÈRE	POINTS
Lecture et nettoyage corrects (« FORCES » et « forces » comptés ensemble)	3
Statistiques et top 5 exacts	3
Erreurs gérées : fichier manquant, argument oublié, fichier vide	3
Rapport lisible et JSON sauvegardé	2
Découpage en modules et en fonctions courtes, bien nommées	3
Au moins 6 tests pytest qui passent	3
README complet, `.gitignore`, commits propres, dépôt public	3

8. Checklist de fin de semaine

- [ ] Je sais écrire, enregistrer et lancer un script depuis le terminal.
- [ ] Je lis une erreur de bas en haut et je la corrige seul.
- [ ] J'écris une condition et une boucle de comptage sans modèle.
- [ ] J'écris une fonction avec `return` et je la teste avec `assert`.
- [ ] Je lis un fichier, je nettoie ses lignes et je gère un fichier manquant.
- [ ] Je sauvegarde un résultat en JSON.
- [ ] Je crée un environnement virtuel et un `requirements.txt`.
- [ ] Je fais `add`, `commit` et `push` sans aide.
- [ ] J'ai un dépôt GitHub public, avec un README et des tests.
- [ ] J'explique chaque ligne de mon mini-projet à voix haute.
